

Draft Interaction Design

State of the Art: Wie moderne Apps und SaaS-Produkte „Entwurf“-Zustände, unvollständige Datensätze und Save-Patterns behandeln. Aufbereitung verifizierter Quellen aus NN/g, Polaris, SLDS, Linear, Pajamas, USWDS u. a. – mit konkreten Empfehlungen für die Pflanzenschutz-Prototypen (Plot-Datensätze).

Kernbefund in einem Absatz

Die explizite **Draft-Markierung ist nicht veraltet** – sie hat sich aber **kontextabhängig ausdifferenziert**. Strukturierte B2B/SaaS-Datensätze (Linear-Issues, Shopify-Produkte, Salesforce-Records) verwenden weiterhin explizite Draft-States mit dedizierten *Badges/Chips*, während kontinuierliche Content-Editoren (Figma, Google Docs, Notion) auf transparenten Autosave mit Versionierung umgestellt haben. Heutiger Best-Practice-Stack: (1) **Save anytime** – Speichern verlangt nur ein Minimum an Pflichtfeldern; (2) **Pflichtfeld-Gating zur Action-Zeit**, nicht beim Feldfokus; (3) **Status-Indikator** als nicht-interaktiver Badge (Informational/Neutral-Tone) statt eines neuen, eigens erfundenen Tones; (4) **Felder NICHT deaktivieren**, sondern inline validieren nach erstem Submit-Versuch.

INHALT

1. Forschungsfragen & Methode
2. Frage 1 – Markiert man heute noch als „Draft“?
3. Frage 2 – Visuelle Signale: Badge, Chip, Pill, Banner, Icon
4. Frage 3 – Autosave-Alternativen und Status-Feedback
5. Frage 4 – Best Practices für unvollständige Datensätze
6. Frage 5 – „Save anytime“ + Gating zur Action-Zeit
7. Frage 6 – Wie machen es Notion, Linear, Figma, Salesforce, GitLab?
8. Konkrete Empfehlung für die Pflanzenschutz-App (Plot-Datensätze)
9. Verifizierte Findings (Übersicht)
10. Widerlegte Annahmen (refuted)
11. Quellen
12. Offene Fragen & Caveats

1. Forschungsfragen & Methode

Sechs Leitfragen wurden in einem Deep-Research-Lauf bearbeitet (104 Subagenten, 22 Quellen gefetched, 83 Claims extrahiert, 25 davon adversarial verifiziert: 16 bestätigt, 9 widerlegt). Bevorzugte

Quellen: Nielsen Norman Group (NN/g), Shopify Polaris, Atlassian Design System, Salesforce Lightning Design System (SLDS), GitLab Pajamas, U.S. Web Design System (USWDS) sowie Engineering-Blogs der untersuchten Produkte (Linear, Figma, Notion).

Frage	Kurzantwort
1 · Markiert man heute noch als „Draft“?	Ja – aber differenziert nach Domäne (strukturierte Records: ja; Content-Editoren: meist nein, ersetzt durch Versionierung)
2 · Welche visuellen Signale?	Badge mit Information/Neutral-Tone (Polaris, Atlassian Lozenge, SLDS); kein eigener „Draft“-Tone
3 · Alternativen zu Draft-Marker?	Autosave-Status („Saving...“ → „Saved 1 min ago“, Pajamas) + Versionierung (Figma) + Completeness-Indikator
4 · Best Practices (NN/g, USWDS)?	Validation zur Action-Zeit, nicht beim Tippen; Felder nicht disable; inline Format-Validierung
5 · „Save anytime“ + Gating?	Save mit minimalen Pflichtfeldern, vollständige Validation erst beim Folge-Aktion (z. B. Publish / Forecast / Submit)
6 · Wie machen es die großen Tools?	Linear: explizite Draft-Persistenz · Notion: kein Top-Level-Draft · Figma: Autosave + Version-Checkpoints · Salesforce: Unsaved-Changes-Dialog · GitLab: Per-Feld-Autosave + Timestamp

2. Frage 1 – Markiert man heute noch Dokumente als „Draft“?

Antwort: Ja, aber kontextabhängig. Die Pattern hat sich in zwei Richtungen ausdifferenziert:

Strukturierte Datensätze → expliziter Draft

Tools wie **Linear, Shopify, Salesforce, Airtable** behandeln einen halbfertigen Record (Issue, Product, Opportunity, Row) als eigenen Lifecycle-Zustand. Begründung: Der Datensatz steht in *Beziehung zu anderen Records* (Reports, Filter, Workflows) – die Umgebung muss wissen, ob er „zählt“.

Linear hat im Mai 2023 explizit ein **persistentes Draft-System** ausgerollt: Beim ESC- oder X-Klick erscheint ein „Save draft“-Prompt, Drafts überleben Logout und sind device-übergreifend.

[Linear-Changelog 2023-05-11]

Content-Editoren → Autosave + Version-History

Figma, Google Docs, Notion haben das klassische Draft-Konzept faktisch *ersetzt*. Statt eines Status „draft“ gibt es nur noch Versionen. Figma erzeugt nach Offline-Edits automatisch *Version-Checkpoints* (vergleichbar mit Git-Commits) vor und nach jedem Merge-Schritt.

[Figma Engineering Blog, April 2024]

Notion verzichtet sogar auf eine Top-Level-Draft-Kategorie: Die *Status-Property* kennt nur drei **nicht umbenennbare** Kategorien – **To Do**, **In Progress**, **Complete**. „Draft“ taucht maximal als User-erstellte Sub-Option auf. [Notion Help: Status property]

Daraus folgt: Die Frage „Brauchen wir noch Drafts?“ ist die falsche Frage. Die richtige lautet: *Wirkt der unvollständige Datensatz auf seine Umgebung? Erscheint er in Listen, Reports, Filtern?* Wenn ja → Draft-Marker. Wenn nein (reiner Editor) → Autosave reicht.

3. Frage 2 – Visuelle Signale: Badge, Chip, Pill, Banner, Icon

Die Mehrheits-Lösung: Badge mit Information-Tone

Quer durch alle untersuchten Design-Systeme wird Draft als **nicht-interaktiver Status-Marker** dargestellt – nicht als Action-Trigger. Drei prägende Vorgaben:

System	Komponente	Konkrete Regel zu Draft
Shopify Polaris	Badge	Kein eigener „Draft“-Tone. Mapping auf <code>tone="info"</code> (Beispiel-Code: <code><s-badge tone="info">Draft</s-badge></code>). Erfinden neuer Tones ist explizit untersagt.
Salesforce SLDS	lightning-badge	„Badges don't work for navigation because they can't include a hyperlink ... Badges with nested elements are not supported.“ → Rein informativ. Für interaktive Labels: <code>lightning-pill</code> .
Atlassian Design	Lozenge	Lozenge ist als <i>Status-Communicator</i> definiert (z. B. issue states wie „In Progress“). Subtle-Variante (default) ist nicht-interaktiv.

Hierarchie der Mittel (Faustregel)

Mittel	Wann?	Beispiel
Badge / Chip	Per-Record-Indikator in Listen, Tabellen, Karten	„Draft“ neben dem Issue-Titel
Lozenge / Pill	Bei Bedarf nach interaktivem Status (klickbar, filterbar)	Jira-Status „In Review“
Banner	Globaler Hinweis innerhalb des geöffneten Records („Diese Version ist ein Entwurf – noch nicht veröffentlicht“)	WordPress Edit-Screen
Icon	Ergänzend in dichten Listen; nie als alleiniges Signal (Accessibility)	Kleines Stift-Icon
Listentrennung	Wenn Draft-Records dominieren (eigene Sektion „Drafts“ oder Tab)	Linear: „Drafts“-Menü im Composer

Anti-Pattern: Eigener „Draft“-Farbton (z. B. rot oder orange) suggeriert Fehler oder Warnung. Polaris und Atlassian raten beide explizit davon ab. Verwende den bestehenden Information/Neutral-Tone des Design-Systems.

4. Frage 3 – Autosave-Alternativen & Status-Feedback

Der GitLab-Pajamas-Standard

Die klarste, am breitesten zitierte Spec stammt von **GitLab Pajamas**. Zwei Zustände, ein Timestamp:

Zustand	Anzeige	Beispiel
Während Save	Saving... + Spinner	Spinner-Icon links neben Text
Nach Save	Saved + relativer Timestamp	Saved just now, Saved 1 min ago
Optional (Draft-Kontext)	Präfix „Draft“	Draft saved 1 min ago

Wichtigste Pajamas-Einschränkung: „Auto-saving should be applied to *each input individually* and *not* to forms as a whole. **Never apply to whole forms.**“ Beispiel aus dem Original: Ein „Personal Access Token“-Formular (Name, Expiration, Scope) soll *nicht* autogespeichert werden – es braucht ein bewusstes Submit.

Versionierung als Recovery-Layer (Figma)

Figmas Engineering-Blog macht transparent, dass Autosave allein das Risiko von Merge-Konflikten erhöht. Lösung: Vor und nach jedem riskanten Schritt (etwa Reconnect nach Offline-Edit) wird automatisch ein *Version-History-Checkpoint* gesetzt. „Saven“ verlagert sich damit nach hinten in die Recovery-Schicht; vorne sieht der User nur noch das durchgehende, transparente Autosave.

Der „Last saved“-Stempel als minimales Vertrauenssignal

Auch bei Autosave verlangt NN/g eine sichtbare Bestätigung – aber differenziert: Für strukturierte Records (Settings, Profile) bleibt der Save-Button als *kognitiver Anker* wertvoll, denn User vermuten ohne ihn fälschlicherweise, dass nichts persistiert ist. Für Editoren ist die Schulung der User auf Autosave (siehe Google Docs) inzwischen abgeschlossen.

5. Frage 4 – Best Practices für unvollständige Datensätze

NN/g: Validation erst beim Submit, nicht beim Tippen

Aus „The Cost of Hostile Patterns in Forms“ (NN/g): Zusätzliche Validierungselemente (rote Border, Warnings, Inline-Errors) sollen erst erscheinen, „*if and after the user attempts to submit a form or page without filling in required fields*“. Vorher genügt der Asterisk am Label. Format-Validation (z. B. E-Mail-Syntax) ist davon getrennt und sollte inline *on blur* stattfinden:

„*Ideally, all validation should be inline; that is, as soon as the user has finished filling in a field.*“ – NN/g, „Errors in Forms“

USWDS: Felder niemals deaktivieren

Das U.S. Web Design System verbietet das Disabling von Form-Inputs zur Progressionssteuerung aus Accessibility-Gründen:

„Don't disable form inputs. Disabled states can confuse users due to low color contrast, lack of meaningful feedback for users of screen readers, and no focus capability for keyboard navigators.“

Statt Felder zu sperren, bis Vorbedingungen erfüllt sind: **Save anytime** erlauben und die Validation am Aktions-Endpunkt platzieren.

Kontrolle vs. Komfort: NN/g „Efficiency vs Expectations“

NN/g formalisiert das Trade-off über das „P-vs-M-Framework“ (Physical vs Mental Effort): „It's not worth reducing P by a small amount if the corresponding M becomes much larger.“ Heißt für Draft-Designs: Wenn das Wegnehmen des Save-Buttons User irritiert, hat der gewonnene Klick keinen Nettowert.

6. Frage 5 – „Save anytime“ + Required Fields zur Action-Zeit

Die saubere Implementierung für strukturierte B2B-Datensätze besteht aus drei Schichten:

Schicht	Was passiert?	Praxis-Beispiel
1. Save-Schwelle (minimal)	Nur 1-3 Felder müssen gesetzt sein, damit der Datensatz überhaupt existiert (typischerweise ein Name/Titel + ggf. ein Owner).	Linear-Issue: Title genügt. Shopify-Product: Title genügt. → Status danach: Draft.
2. Action-Schwelle (vollständig)	Vor jeder Folge-Aktion (Publish, Submit, Forecast, Send) prüft das System gegen das volle Pflichtfeld-Set und gibt dem User dosiert die Möglichkeit, die Lücken zu schließen.	WordPress: „Publish“ zeigt fehlende Felder. Shopify: „Activate Product“ listet fehlende Anforderungen.
3. Recovery / Schutz	Unsaved-Changes-Dialog beim Verlassen; bei kollaborativen Tools zusätzlich Versionierung.	Salesforce <code>lightning:unsavedChanges</code> : Save/Discard-Dialog beim Tab-Schließen.

Designprinzip: Pflichtfelder sind *kein Save-Gate*, sondern ein *Action-Gate*. Save = „ich will mir das merken“. Action = „ich will dem System sagen, dass es jetzt etwas damit tun soll“. Diese Trennung kommt direkt aus dem NN/g-Validation-Modell und ist die theoretische Basis für „Save anytime“.

7. Frage 6 – Wie machen es Notion, Linear, Figma, Salesforce, GitLab?

Linear · Persistente, geräteübergreifende Drafts

Beim ESC oder X-Klick im New-Issue-Composer erscheint ein „Save draft“-Prompt. Titel, Description, Properties werden persistiert. „Saved drafts will persist on logout and are available across devices.“ Zugriff über einen dedizierten *Drafts*-Button im Composer, der ein Menü mit allen gespeicherten Drafts öffnet.

Quelle: linear.app/changelog/2023-05-11-issue-drafts · Verifikation: 3-0

Notion · Drei fixe Status-Kategorien, kein Top-Level-Draft

Die Status-Property kennt nur `To Do` / `In Progress` / `Complete`. „You can't change the three main categories.“ „Draft“ gilt als User-erstellte Sub-Option innerhalb von „In Progress“ (Beispiele aus Notion-Doku: „1st Draft, 2nd Draft, Editing“). Damit explizite Abkehr von Draft als globalem Status.

Quelle: notion.com/help/guides/status-property-gives-clarity-on-tasks · Verifikation: 3-0

Figma · Autosave + automatische Version-Checkpoints

Continuous Autosave; Risiko von Merge-Konflikten nach Offline-Sessions wird über automatisch erzeugte Version-History-Checkpoints (Git-Commit-Analogie) entschärft. „We automatically create a version history checkpoint before and after applying the diff.“ User können beim Konflikt zurückrollen.

Quelle: figma.com/blog/behind-the-feature-autosave/ · Verifikation: 3-0

Salesforce · Unsaved-Changes-Dialog statt Silent-Autosave

SLDS stellt `lightning:unsavedChanges` bereit: Eine Komponente, die der UI-Containment-Hierarchy ungesicherte Änderungen signalisiert. Bei Aktionen, die Datenverlust bedeuten (z. B. Console-Tab schließen), erscheint ein Save/Discard-Dialog. SLDS-Save-Workflow setzt damit explizit auf bewusste Bestätigung, nicht auf stilles Background-Save.

Quelle: developer.salesforce.com/docs/platform/lightning-component-reference/guide/aura-unsaved-changes.html · Verifikation: 3-0

GitLab Pajamas · Two-State-Indikator, Per-Feld-Autosave

`Saving...` mit Spinner während Save, danach `Saved just now` / `Saved 1 min ago`. Mit Draft-Kontext optional `Draft saved 1 min ago`. Wichtigste Regel: „Auto-saving should be applied to each input individually and not to forms as a whole. Never apply to whole forms.“

Quelle: design.gitlab.com/patterns/saving-and-feedback/ · Verifikation: 3-0

Shopify Polaris · Draft = Informational Badge

Kein eigener Draft-Tone. Mapping auf `tone="info"` über die existierende Badge-Komponente: `<s-badge tone="info">Draft</s-badge>`. Erfinden neuer Tones ist explizit untersagt – Begründung: Konsistenz und Accessibility.

Quelle: polaris-react.shopify.com/components/feedback-indicators/badge · Verifikation: 3-0

8. Konkrete Empfehlung für die Pflanzenschutz-App (Plot-Datensätze)

Die Pflanzenschutz-App hat ein typisches Profil für *explicit Draft + Save anytime*: Plot-Datensätze stehen in Beziehung zu Treatments, Lab-Reports und Forecasts. Sie erscheinen in Listen, Filtern und Reports. Damit qualifizieren sie sich klar für die **strukturierte Variante**, nicht für reine Editor-Autosave-Logik.

Empfohlenes Pattern (4 Schichten)

Schicht	Konkrete Umsetzung	Begründung
Save-Schwelle	Minimum (z. B. Plot-Name) – evtl. zusätzlich nicht-zukünftige Felder (Crop, Variety, Location), aber NICHT die beiden Datums-Felder (Planting, Flowering).	Datums-Felder beziehen sich auf vergangene/zukünftige Ereignisse, die der User bei Plot-Anlage oft noch nicht kennt. Save-Gating darauf erzeugt Friktion ohne Nutzen.
Listen-Marker	Badge „Draft“ neben dem Plot-Namen. Tone = informational/neutral des Design-Systems. Nicht interaktiv. Keine eigene Farbe.	Polaris/SLDS/Atlassian-Konvention. Konsistenz mit etablierten Patterns reduziert Lernkost für User, die andere SaaS-Tools kennen.
Action-Gating	Vor „Get Forecast“ prüfen, ob Planting- und Flowering-Date gesetzt sind. Bei Lücke: kleines Inline-Dialog mit genau diesen 2 Feldern. (Genau das, was in v6 schon implementiert ist.)	NN/g: Validierung zur Action-Zeit. Pajamas: niemals ganze Forms auf einmal validieren. → Nur die für die Folge-Aktion benötigten Felder erzwingen.
Recovery	Unsaved-Changes-Dialog beim Verlassen der Add/Edit-Seite, wenn ungesicherte Änderungen existieren. „Save draft / Discard / Cancel“.	Salesforce-Pattern. Verhindert Datenverlust ohne den User zu Autosave zu zwingen.

Was zu vermeiden ist

- **Felder bei „Plot ist Draft“ disabled rendern.** Verstoß gegen USWDS-Regel. Stattdessen: alle Felder editierbar, Validation am Action-Punkt.
- **Eigene rote/orange „Draft“-Badge-Farbe.** Suggestiert Fehler. Informationale-Tone des Design-Systems verwenden.

- **Autosave für die gesamte Plot-Anlage-Maske.** Pajamas-Verbot. Plot-Anlage ist ein strukturiertes Submit, kein Editor.
- **Pflichtfeld-Validation beim ersten Tippen.** NN/g-Regel: Errors erst nach Submit-Versuch, Format-Errors inline on blur.

Direkte Konsequenz für v6: Die in v6 gewählte Lösung – „nur Planting- und Flowering-Date sind beim Save optional, kleines Dialog vor Forecast“ – ist *state of the art*. Was die Recherche zusätzlich nahelegt: (a) den Draft-Badge Tone auf den Information-Tone des verwendeten Material-Systems setzen statt rot; (b) den „Unsaved Changes“-Schutz beim Verlassen der Add-Plot-Maske hinzufügen; (c) für künftige V7-Iteration die Kombination aus Per-Feld-Autosave (à la Pajamas) auf einzelnen Feldern und explizitem Save für den Datensatz prüfen.

9. Verifizierte Findings (Übersicht)

1. Explizite Draft-Markierung ist in B2B-SaaS weiterhin Standard – als nicht-interaktiver Status-Marker (Badge), nicht als Action-Trigger. HIGH

Belege: Polaris erlaubt kein Erfinden neuer Tones, mappt Draft auf Information. SLDS-Badges sind ohne Hyperlinks/Nested Elements rein informativ. Linear hat explizites Draft-Konzept im Composer.

Quellen: polaris-react.shopify.com · developer.salesforce.com · linear.app/changelog · Vote: 3-0

2. Linear implementiert Drafts als persistenten, geräteübergreifenden Zustand mit explizitem „Save draft“-Dialog beim Schließen/ESC. HIGH

Drafts persistieren über Logout hinweg, sind device-übergreifend verfügbar. Zugriff über dedizierten Drafts-Button im Composer.

Quelle: linear.app/changelog/2023-05-11-issue-drafts · Vote: 3-0

3. GitLab Pajamas definiert State-of-the-Art Autosave-Status-Indikator: zwei Zustände, nur auf Einzelfeld-Ebene. HIGH

`Saving...` + Spinner → `Saved` + relativer Timestamp. Niemals ganze Forms auto-speichern.

Quelle: design.gitlab.com/patterns/saving-and-feedback/ · Vote: 3-0

4. Salesforce schützt Draft-Zustände über Confirmation-Dialog beim Navigation/Close, nicht über Silent-Autosave. HIGH

`lightning:unsavedChanges` -Komponente signalisiert ungesicherte Inhalte und triggert Save/Discard-Dialog.

Quelle: developer.salesforce.com/docs/.../aura-unsaved-changes.html · Vote: 3-0

5. Pure Autosave ohne Save-Button erhöht kognitive Last in strukturierten Formularen (kontextabhängig). **MEDIUM**

NN/g P-vs-M-Framework: weniger Steps physisch, mehr Aufwand mental. Gilt für strukturierte B2B-Records, nicht für kollaborative Content-Editoren.

Quelle: nngroup.com/articles/efficiency-vs-expectations/ · Vote: 2-1 (split)

6. Figma kombiniert kontinuierliches Autosave mit automatischen Versions-Checkpoints.

HIGH

Bei Reconnect nach Offline-Sessions: Version-History-Checkpoint vor und nach Merge erzeugt – wie Git-Commit. User können bei Konflikt rückrollen.

Quelle: figma.com/blog/behind-the-feature-autosave/ · Vote: 3-0

7. Notion verzichtet auf Top-Level-Draft-Status; fix drei Kategorien: To Do / In Progress / Complete. **HIGH**

„You can't change the three main categories.“ Draft tritt nur als User-erstellte Sub-Option auf.

Quelle: notion.com/help/guides/status-property-gives-clarity-on-tasks · Vote: 3-0

8. Required-Field-Validierung gehört zur Action-Zeit (Submit), nicht zum Feldfokus. **HIGH**

NN/g „Hostile Patterns“: Rote Borders, Inline-Errors erst nach Submit-Versuch. Pflichtfelder vorab via Asterisk/Label kennzeichnen. Format-Validation (E-Mail-Syntax) separat inline on blur.

Quellen: nngroup.com/articles/hostile-error-messages/ · nngroup.com/articles/errors-forms-design-guidelines/ · Vote: 3-0 / 2-1

9. USWDS verbietet das Disabling von Formular-Inputs zur Progressionssteuerung (Accessibility). **HIGH**

„Don't disable form inputs.“ Stattdessen Error-Messages und Inline-Validation für Korrektur.

Quelle: designsystem.digital.gov/patterns/complete-a-complex-form/progress-easily/ · Vote: 3-0

10. Autosave ist die richtige Wahl, wenn Content-Fokus wichtiger ist als explizite Kontrolle.

MEDIUM

UI-Patterns: Für lange Texte / kreative Arbeit Autosave; für strukturierte Daten mit klaren Submit-Semantiken explizites Save + Draft-Marker.

Quelle: ui-patterns.com/patterns/autosave · Vote: 3-0 (secondary source)

10. Widerlegte Annahmen (refuted)

Die adversariale Verifikation hat 9 Claims verworfen. Hier die wichtigsten – als **Negativ-Liste**, damit nicht versehentlich darauf aufgebaut wird:

NN/g unterscheide explizit drei UI-Communication-Methoden: Indicators, Validations, Notifications. REFUTED 0-3

Die Behauptung ließ sich im Original-Artikel nicht in dieser Schärfe belegen. Die *praktische* Anwendung (Draft = Indicator) bleibt aber via Polaris/SLDS/Linear belegt.

USWDS empfehle Save-and-Resume als essenziell für komplexe Formulare. REFUTED 0-3

USWDS empfiehlt zwar Progressive Disclosure und keine disabled Inputs – Save-and-Resume wird im Original aber nicht so klar als Pflicht beschrieben.

Auch mit Autosave sollte ein Save-Button erhalten bleiben, weil User „panicken“ ohne ihn.

REFUTED 0-3

In dieser absoluten Form nicht aus der Primärquelle belegbar. Kontextabhängig korrekt für B2B, aber nicht universell.

Visuelles Autosave-Feedback durch dynamisches Relabeling des Save-Buttons („Save“ → „Saved“). REFUTED 0-3

Pattern existiert in der Praxis, aber als universelle Empfehlung nicht aus den geprüften Quellen ableitbar. Pajamas-Pattern (separates Status-Label mit Timestamp) ist robuster.

Die Mehrheit der User nimmt an, dass Änderungen erst nach Save-Klick persistiert sind.

REFUTED 1-2

In dieser Allgemeinheit widerlegt – User-Erwartungen sind über die letzten Jahre Richtung Autosave gewandert (Google Docs, Notion). Für strukturierte B2B-Records bleibt der Save-Erwartungsanker stark, ist aber kein universelles Gesetz mehr.

11. Quellen (geprüft)

PRIMÄRQUELLEN (DESIGN SYSTEMS & ENGINEERING BLOGS)

- Shopify Polaris – Badge: <https://polaris-react.shopify.com/components/feedback-indicators/badge>
- Salesforce SLDS – lightning-badge: <https://developer.salesforce.com/docs/platform/lightning-component-reference/guide/lightning-badge.html>
- Salesforce SLDS – Unsaved Changes: <https://developer.salesforce.com/docs/platform/lightning-component-reference/guide/aura-unsaved-changes.html>
- Atlassian Design – Lozenge: <https://atlassian.design/components/lozenge>
- Atlassian Design – Badge Usage: <https://atlassian.design/components/badge/badge/usage>

- GitLab Pajamas – Saving and Feedback: <https://design.gitlab.com/patterns/saving-and-feedback/>
- U.S. Web Design System – Complete a Complex Form: <https://designsystem.digital.gov/patterns/complete-a-complex-form/progress-easily/>
- Linear Changelog – Issue Drafts (2023-05-11): <https://linear.app/changelog/2023-05-11-issue-drafts>
- Figma Engineering Blog – Behind the Feature: Autosave (2024): <https://www.figma.com/blog/behind-the-feature-autosave/>
- Notion Help – Status Property: <https://www.notion.com/help/guides/status-property-gives-clarity-on-tasks>

NIELSEN NORMAN GROUP (NN/G)

- Hostile Patterns in Forms (Error Messages): <https://www.nngroup.com/articles/hostile-error-messages/>
- Errors in Forms – Design Guidelines: <https://www.nngroup.com/articles/errors-forms-design-guidelines/>
- Efficiency vs Expectations: <https://www.nngroup.com/articles/efficiency-vs-expectations/>
- Indicators, Validations, Notifications: <https://www.nngroup.com/articles/indicators-validations-notifications/> (kritisch zu lesen – einzelne Claims widerlegt)

SEKUNDÄRQUELLEN (SAMMELSEITEN, BLOGS)

- Smart Interface Design Patterns – Badges, Chips, Tags, Pills: <https://smart-interface-design-patterns.com/articles/badges-chips-tags-pills/>
- UI-Patterns – Autosave: <https://ui-patterns.com/patterns/autosave>
- Damian Wajer – Autosave Patterns: <https://www.damianwajer.com/blog/autosave/>
- UXPin Studio Blog – Progressive Disclosure: <https://www.uxpin.com/studio/blog/what-is-progressive-disclosure/>

12. Offene Fragen & Caveats

WAS DIE RECHERCHE NICHT ABSCHLIESSEND BELEGT

- Konkrete Empfehlungen von **Material Design 3**, **Carbon (IBM)**, **Atlassian** spezifisch zum Draft-State wurden im Verifikations-Sample nicht vollständig abgedeckt, obwohl Lozenge/Badge dort dokumentiert sind. Lohnt einen Direkt-Lookup im jeweiligen System vor Implementierung.
- Quantitative Effekte von Autosave auf User-Errors bei strukturierten B2B-Datensätzen vs. Editoren – keine harten Zahlen (z. B. von Baymard) im Lauf gefunden.
- Airtable und Microsoft 365 (Word/Excel Online) – ihre konkrete Visualisierung von Draft- vs. Published-Records in gemischten Listen wurde nicht vertieft.
- Accessibility-Anforderungen für Saving/Saved-Status-Announcements (WCAG 2.2, ARIA-Live-Regions) bleiben ein eigenes Recherche-Thema.

METHODISCHE CAVEATS

- Adversarial Vote (3-Vote-Verifikation): Aussagen wurden als „bestätigt“ gewertet, wenn $\geq 2/3$ Verifier sie nicht refutieren konnten. Das ist robust gegen einzelne falsch-positive Stellen, aber kein Ersatz für Domänen-Expertise.
- Linear-Feature ist Mai 2023, Pajamas/SLDS/Polaris werden laufend aktualisiert – Detail-Wording kann sich in 2026 leicht verschoben haben. Vor Implementierung Stand prüfen.
- Salesforces `lightning:unsavedChanges` ist Aura-Legacy; das LWC-Äquivalent existiert, ist aber organisatorisch anders – API-Cross-Check empfohlen.

Forschungsbericht generiert via Deep-Research-Workflow (5 Such-Angles, 104 Subagenten, 22 Quellen, 25 verifizierte Claims). Zusammenstellung und deutsche Synthese: *für Lyle Peterer · ResiYou-Variations Prototyp, Pflanzenschutz-App · Juni 2026.*